

Kernels

© Copyright 2021 Fusemachines, Inc. | All rights reserved

Prerequisites

Students must know of the following topics:

- Concepts of Linear Classifier, ML level 1 and 2

Learning Objectives

After reading this notebook, students will be able to:

- Understand linearly separable and non-linearly separable data.
- Understand the concept of data transformation.
- Understand the concept of kernels.
- Create a new kernel

Introduction

Up to this point, we all know how linear classification works. It is also important that for a precise classification we need to have a linearly separable data.

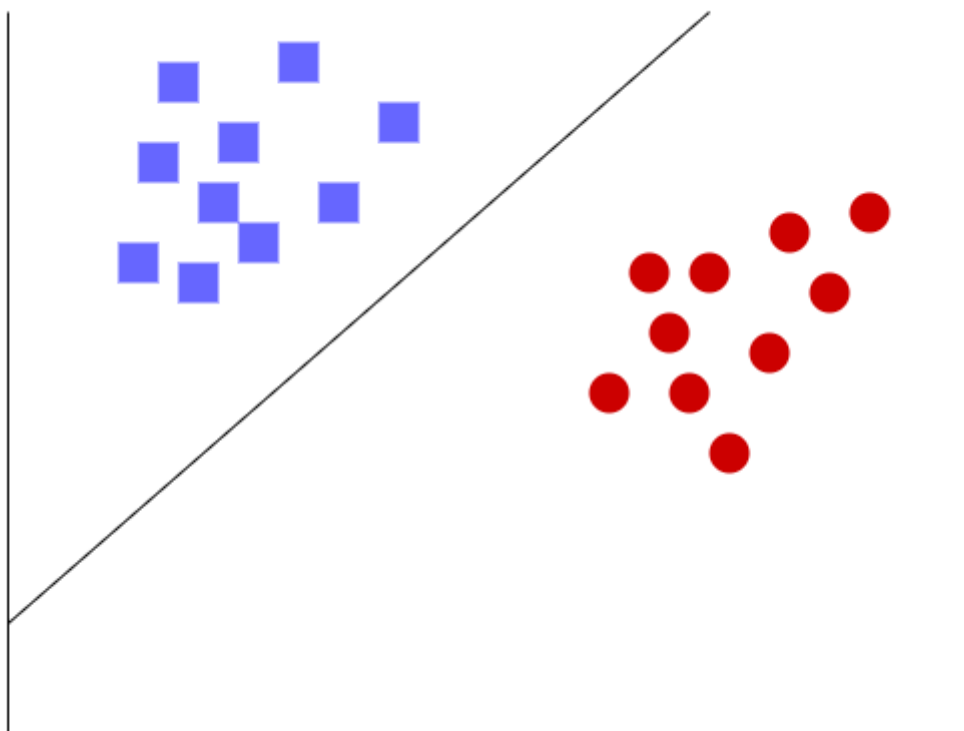


Fig 1: Linearly separable data

But what if do not have a linearly separable data and we have to do linear classification on that data like shown below.

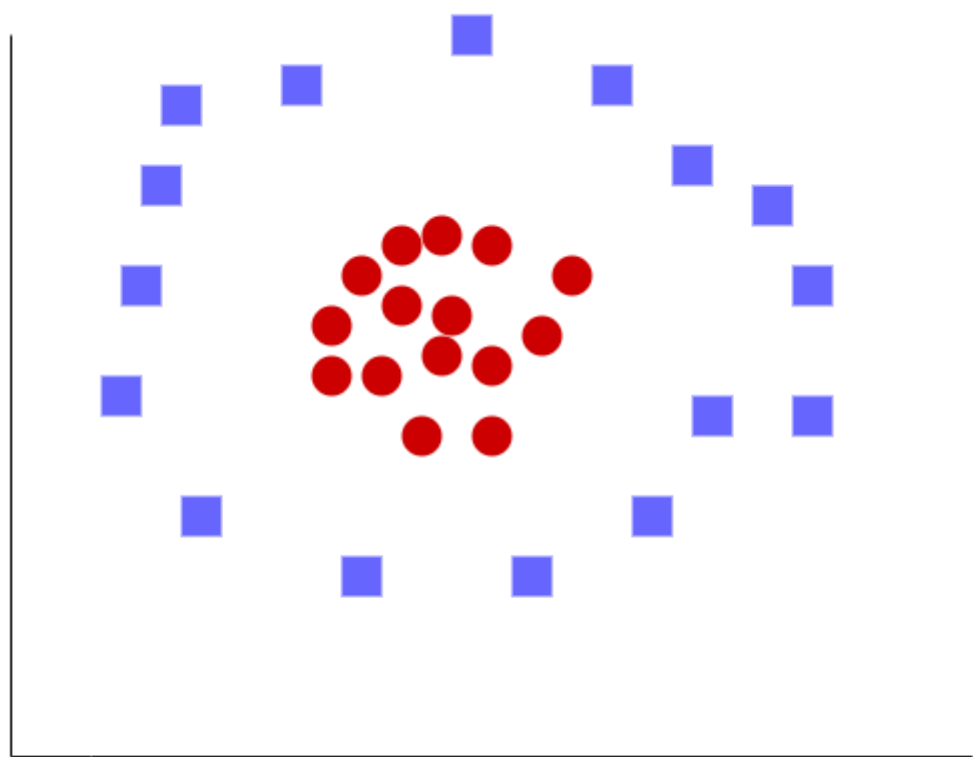


Fig 2: Linearly inseparable data

For example, let us talk about what is commonly known as **XOR** problem. We are going to pass all the combinations of values for features $x_1 \in \{0, 1\}$ and another feature $x_2 \in \{0, 1\}$. The output of the combination of the elements from the data points is shown in the table below.

x_1	x_2	y
1	1	0
1	0	1
0	1	1
0	0	0

Table 1: XOR function table

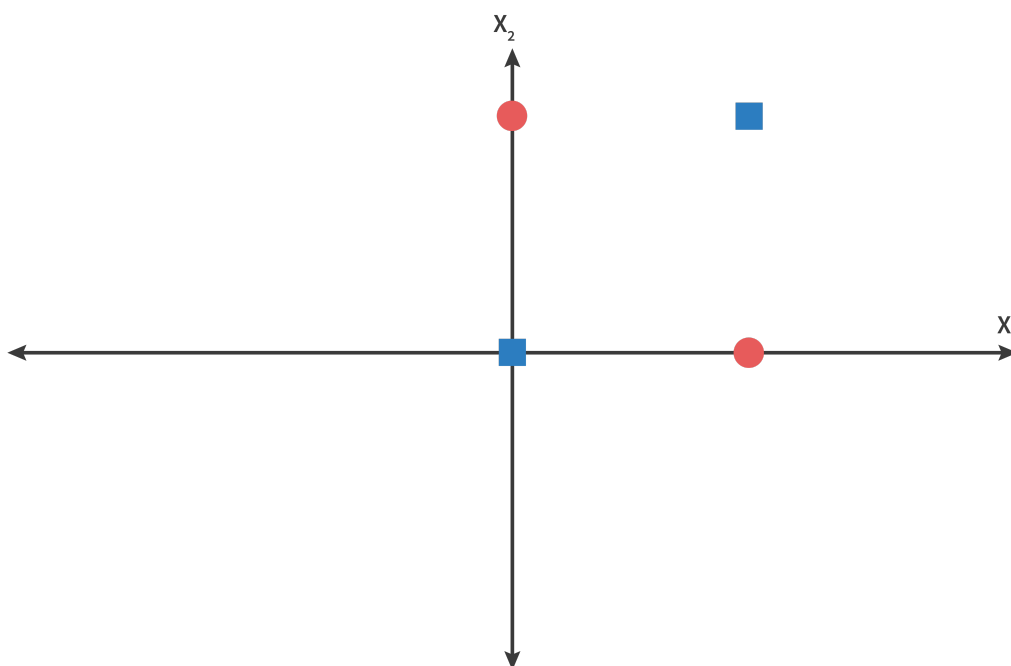


Fig. 3: Plot of XOR for two features

In the above figure, the class of zeros is represented by square and class of one is represented by the circle. We can see on the figure that we cannot solve the above condition using a linear model(single model).

XOR classification is unsolvable using a perceptron for the case we have seen above. But, there are methods to solve that type of problem. Especially two of them are:

- Using multiple linear models
- Transforming the data

Data Transformation

It is the most common technique used in non-linear data transformation. Data transformation is the technique of adding new data feature into the training data just by repurposing the existing data.

The idea behind data transformation is that, if we transform data to increase the dimensions without altering the original data, the separation between the data distribution becomes prominent and it becomes easier for us to classify accordingly.

In order to understand the idea behind data transformation , Let's imagine we have a function $\phi(x_i)$ that performs a transformation of the feature x_i . The feature transformation is performed by creating various combination of existing features. For example, when we have a data x with three features represented as,

$$x = \begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix}. \text{ The transformation of the feature would be}$$

$$\phi(x) = \begin{bmatrix} 1 \\ x_1 \\ x_2 \\ x_3 \\ x_1 x_2 \\ x_1 x_3 \\ x_2 x_3 \\ x_1 x_2 x_3 \end{bmatrix}$$

. A single transformed feature in the new feature space can be thought of as either taking an old feature into computation or not. For example, the first term 1 of this transformed feature ($\phi(x)$) is obtained by considering none of the features of x . On the other hand, the last term $x_1 x_2 x_3$ is considering all the features of x . With this kind of transformation, the number of features in the transformed feature space is 2^n , where n is the dimension of vector x .

For a more general case, if we have a feature vector

$$X = \begin{bmatrix} x_1 \\ x_2 \\ x_3 \\ \vdots \\ x_d \end{bmatrix}$$

, then

$$\phi(x) = \begin{bmatrix} 1 \\ x_1 \\ x_2 \\ \vdots \\ x_d \\ x_1 * x_2 \\ x_1 * x_3 \\ \vdots \\ x_1 * x_d \\ \vdots \\ x_1 * x_2 * \cdots * x_d \end{bmatrix}$$

The number of elements in $\phi(x)$ are in the range of 2^d as shown in the above example. The size of $\phi(x)$ increases exponentially with the dimension of the data x_i . In the general case, the size of x_i will be in the range of hundreds and sometimes even thousands. When the size of x_i is large, calculating $\phi(x_i)$ becomes computationally infeasible. It might be confusing to you why are we talking about the transformation which in the general case is not feasible to do. We are going to talk about this idea throughout this chapter and build upon it. Actually, we are also going to see a function which will transform our data to infinite dimension, which we can see is not computable even if we increase our computational resources.

You might be wondering how can we perform the data transformation to the high dimension, even to the infinite dimension? Here comes a beautiful technique which we will cover in detail below and is called the **Kernel Trick**. Kernel trick allows us to transform our features to a higher feature dimension and back to the original

dimension by only computing the inner product of data points. If it is unclear to you, do not be intimidated. After going through this chapter, you will have a thorough understanding of the concept as well as the mathematical derivation of the kernel trick.

Let's implement the ideas discussed above by using a practical example. Here we will be solving the *XOR* classification and circular data classification.

In []:

```
%matplotlib inline
import numpy as np
import matplotlib.pyplot as plt
from mpl_toolkits.mplot3d import Axes3D

from sklearn.linear_model import Perceptron

x1 = np.array([1, 1, 0, 0])
x2 = np.array([1, 0, 1, 0])
x3 = np.array([1, 0, 0, 0])
y = np.array([0, 1, 1, 0])

# stack the features x1, x2 and x3 to get a feature matrix X
X = np.stack((x1, x2, x3), axis=1)

# define a classifier from sklearn library
clf = Perceptron(tol=1e-3, random_state=2)

# fit the classifier to our datapoints
clf.fit(X, y)

# get a single parameter vector by stacking the intercept and coefficients
_w = np.hstack((clf.intercept_, clf.coef_.squeeze()))
print(clf.intercept_, clf.coef_, _w)

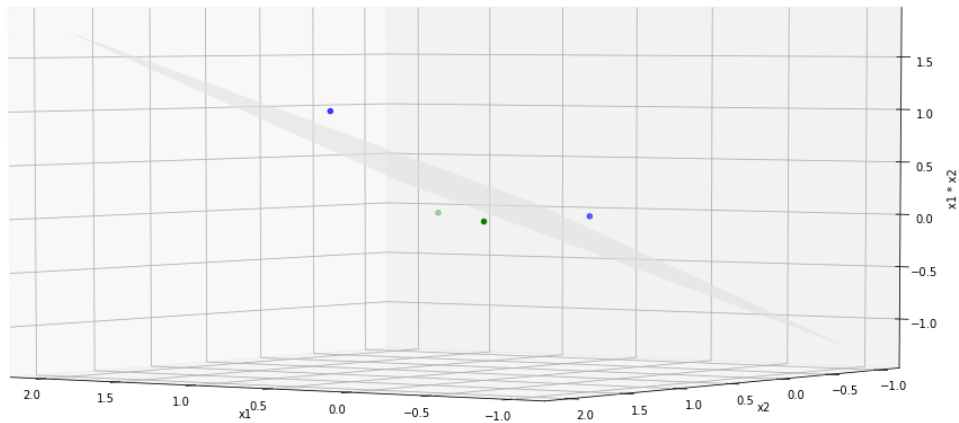
xx1, xx2 = np.meshgrid(range(-1, 3), range(-1, 3))
xx3 = -(_w[0] + _w[1] * xx1 + _w[2] * xx2) / _w[3]
# print(xx1.shape, xx2.shape)

fig = plt.figure(figsize=(14, 8))
ax = Axes3D(fig)

ax.set_xlabel('x1')
ax.set_ylabel('x2')
ax.set_zlabel('x1 * x2')

# help(ax.scatter)
ax.scatter(x1, x2, x3, c=['g' if y_i == 1 else 'b' for y_i in y])
ax.plot_surface(xx1, xx2, xx3, color='#f0f0f0', alpha=0.6)
ax.view_init(elev=3, azimuth=125)
plt.show()
```

```
[-1.] [[ 2.  2. -4.]] [-1.  2.  2. -4.]
```



Circular data transformation

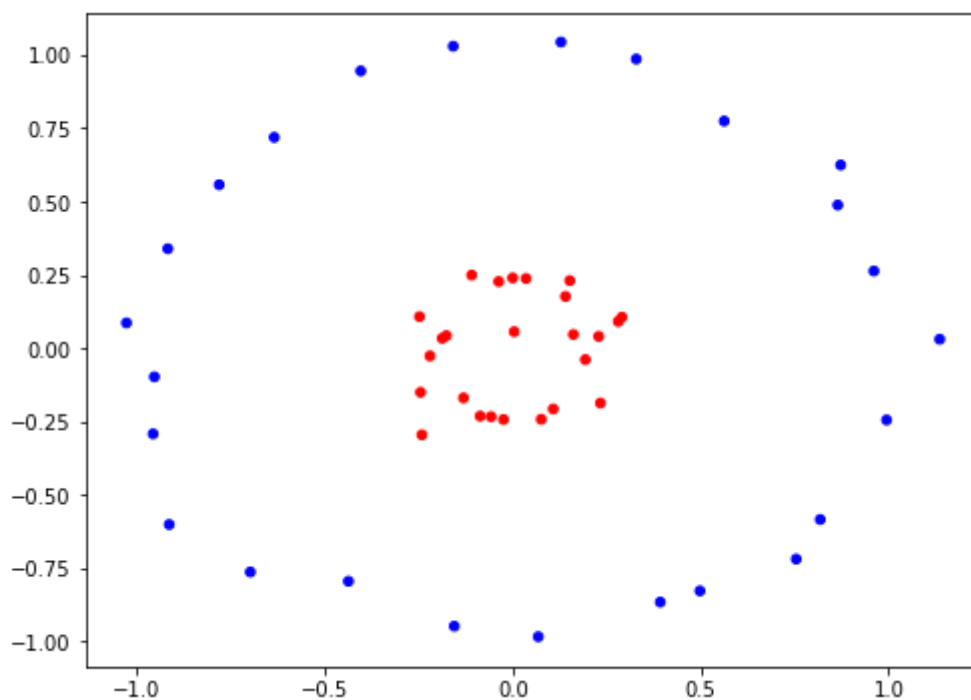
In []:

```
from sklearn.datasets import make_circles
X, y = make_circles(50, noise=0.05, random_state=17, factor=0.25)

plt.figure(figsize=(8,6))
colors = ['r' if y_i == 1 else 'b' for y_i in y]
plt.scatter(X[:, 0], X[:, 1], s=20, c=colors)
```

Out[2]:

<matplotlib.collections.PathCollection at 0x7ffb37745898>



In []:

```
from sklearn.linear_model import Perceptron

X_n2 = np.sum(np.square(X), axis=1)

fig = plt.figure(figsize=(14,6))
ax1 = fig.add_subplot(121, projection='3d')
ax2 = fig.add_subplot(122, projection='3d')

ax1.scatter(X[:, 0], X[:, 1], X_n2, c=colors)

clf = Perceptron(random_state=8)
clf.fit(np.array([X[:, 0], X[:, 1], X_n2]).T, y)

_w = np.hstack((clf.intercept_, clf.coef_.squeeze()))
xx1, xx2 = np.meshgrid(np.arange(-2, 2, 0.2), np.arange(-2, 2, 0.2))
print(_w)
xx3 = (_w[0] + _w[1] * xx1 + _w[2] * xx2) * (-1/_w[3])
ax1.plot_surface(xx1, xx2, xx3, color='magenta', alpha=0.3)

ax2.scatter(X[:, 0], X[:, 1], X_n2, c=colors)

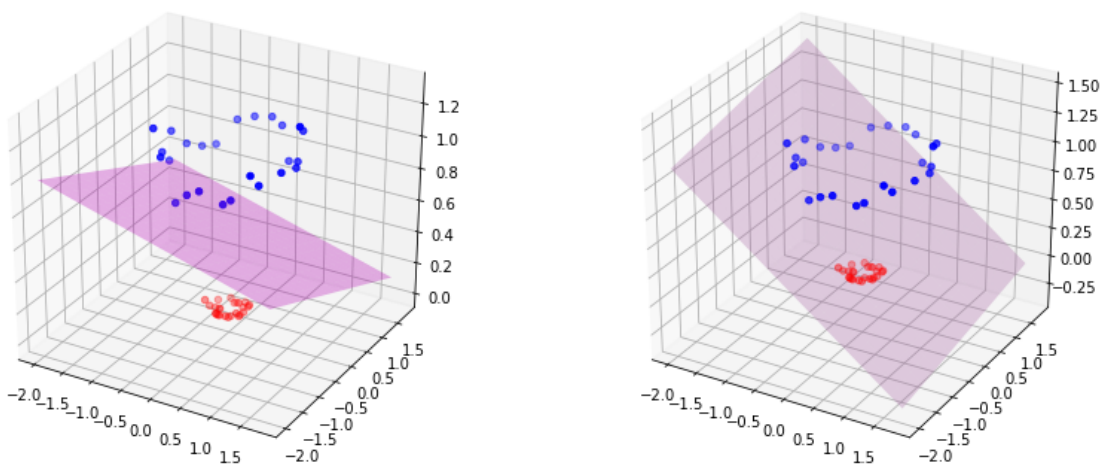
clf = Perceptron(random_state=12)
clf.fit(np.array([X[:, 0], X[:, 1], X_n2]).T, y)

_w = np.hstack((clf.intercept_, clf.coef_.squeeze()))
xx3 = (_w[0] + _w[1] * xx1 + _w[2] * xx2) * (-1/_w[3])
ax2.plot_surface(xx1, xx2, xx3, color='#f28fee', alpha=0.3)
```

```
[ 1.          -0.19157204 -0.23548906 -1.83525446]
```

Out[3]:

<mpl_toolkits.mplot3d.art3d.Poly3DCollection at 0x7ffb37686cf8>



From the above plots(XOR and Circular data classification), it is clear that a hyperplane can easily separate the two classes represented by red and blue dots. Before adding the new feature, our dataset was harder to separate. But the addition of a new feature made the task of finding a separating hyperplane easier or even possible for some cases.

Kernel Trick

Before diving into the kernel trick, we must know what a kernel is? After getting an intuition of a kernel then only we can understand kernel trick better.

Simply speaking, kernel is just a mapping function. Kernel function maps the given data into higher dimensional data. For the given datapoint x and z , we represent kernel function(K) as :

$$k(x_1, x_2) = \langle \phi(x_1), \phi(x_2) \rangle$$

Here, x_1 and x_2 are the given datapoints, $\phi(x_1)$ and $\phi(x_2)$ are the complex non linear values.

To understand the mathematics and intuition of kernels, lets start with a simple linear regression. $y = w^T x$
After we add composite features into the data, And our equation seems like:

$$y = w^T \phi(x)$$

but, how to find ϕ i.e. $\phi(x) = ?$

Lets take the cost function for our linear regression model.

$$J(w) = \frac{1}{2} \sum_{i=1}^n (y_n - w^T x_n)^2$$

for optimum weight, we want:

$$w^* = \operatorname{argmin} J(w)$$

After vectorizing and simplifying we get,

$$w = (x^T x)^{-1} x^T y$$

For complex features, we can write above expression as:

$$w = (\phi^T \phi)^{-1} \phi^T y$$

Now lets get into the **Kernalization**. As we have our cost function for the linear regression:

$$J(w) = \frac{1}{2} \sum_{i=1}^n (y_n - w^T \phi(x_n))^2$$

By taking derivative with respect to weight w , we can solve for w ; w^* means optimum weight.

$$w^* = \sum_{i=1}^n (y_n - w \phi(x_n)) \phi(x_n)$$

consider, $\alpha_n = \sum_{i=1}^n (y_n - w \phi(x_n))$ (This is just to make the equation easier)

$$w^* = \sum_{i=1}^n \alpha_n \phi(x_n)$$

Therefore, $w^* = \phi^T \alpha$

Revisiting our old equation again:

$$J(w) = \sum_{i=1}^n (y_n - w^T \phi(x_n))^2$$

We now vectorize and substitute our weight w and we keep simplifying our expression.

$$J(w) = (y - \phi w)^T (y - \phi w)$$

$$J(w) = y^T y - y^T \phi w - w^T \phi^T y + w^T \phi^T \phi w$$

$$J(\alpha) = y^T y - y^T \phi (\phi^T \alpha) - (\phi^T \alpha)^T \phi^T y + (\phi^T \alpha)^T \phi^T \phi (\phi^T \alpha)$$

$$J(\alpha) = y^T y - y^T \phi \phi^T \alpha - \alpha^T \phi \phi^T y + \alpha^T \phi \phi^T \phi \phi^T \alpha$$

Here $\phi \phi^T$ is a Gram Matrix or Kernel Matrix. We will denote it as K from this point.

$$K = \begin{bmatrix} \phi(x_1)^T \phi(x_1) & \cdots & \phi(x_1)^T \phi(x_n) \\ \vdots & \ddots & \vdots \\ \phi(x_m)^T \phi(x_1) & \cdots & \phi(x_m)^T \phi(x_n) \end{bmatrix}$$

The Gram matrix has a element that are the dot product of the every pair of feature vectors and it exhibits the following properties:

- Symmetry:

$$K^T = K$$

- Positive Semi Definite:

$$A^T K A \geq 0 \text{ i.e. product to any vector}(A) \text{ and it's transpose is always positive.}$$

Continuing with our equation,

$$J(\alpha) = y^T y - y^T \phi \phi^T \alpha - \alpha^T \phi \phi^T y + \alpha^T \phi \phi^T \phi \phi^T \alpha$$

Simplifying the above equation,

$$J(\alpha) = y^T y - y^T K \alpha - \alpha^T K y + \alpha^T K K \alpha$$

$$J(\alpha) = y^T y - y^T K \alpha - y^T K \alpha + \alpha^T K^2 \alpha \quad (\because K = K^T)$$

$$J(\alpha) = y^T y - 2y^T K \alpha + \alpha^T K^2 \alpha$$

To get the optimum value of α , we have to minimize the function. We can do that simply by taking derivatives with respect to α . Hence our equation will be:

$$\frac{\delta J(\alpha)}{\delta \alpha} = -2y^T K + 2\alpha^T K^2 = 0$$

$$-2y + 2\alpha^T K = 0$$

$$\alpha^T = y^T K$$

$$\alpha = K^{-1} y$$

Once we get the optimal α , We can express the weights in the terms of kernel matrix:

$$w = \phi^T \alpha = \phi^T K^{-1} y = \phi(\phi \phi^T)^{-1} y$$

Now lets have a look at the equation that we derived before without kernels: Without Kernels:

$$w = (\phi^T \phi)^{-1} \phi^T y,$$

$$\text{With Kernels: } w = \phi(\phi \phi^T)^{-1} y$$

What is the difference? Before we needed the $\phi^T \phi$ which is the covariance matrix but the $\phi \phi^T$ is the kernel matrix. I may look like we still need ϕ . As we discussed before $\phi \phi^T$ is a kernel matrix. being a kernel matrix, it is symmetrical and it is positive semi-definite.

If look into the mercer's theorem, It says that a symmetrical positive semi definate metrix can be written as the dot product of ϕ i.e.

$$\phi \phi^T = K = \begin{bmatrix} \phi(x_1)^T \phi(x_1) & \cdots & \phi(x_1) \phi(x_n) \\ \vdots & \ddots & \vdots \\ \phi(x_m)^T \phi(x_1) & \cdots & \phi(x_m)^T \phi(x_n) \end{bmatrix} = \begin{bmatrix} k(x_1, x_1) & \cdots & k(x_1, x_n) \\ \vdots & \ddots & \vdots \\ k(x_m, x_1) & \cdots & k(x_m, x_n) \end{bmatrix}$$

This is the infamous Kernel Trick

Lets look at an example. Consider a valid kernel:

$$k(x, z) = (x^T z)^2$$

Suppose, we take two elements in our data and hence our datapoint becomes $x = (x_1, x_2)$ and $z = (z_1, z_2)$.

Now we expand the above equation,

$$k(x, z) = (x^T z)^2 = (x_1 z_1 + x_2 z_2)^2$$

$$k(x, z) = (x_1 z_1)^2 + 2x_1 z_1 x_2 z_2 + (x_2 z_2)^2$$

$$k(x, z) = (x_1^2, \sqrt{2}x_1 x_2, x_2^2)(z_1^2, \sqrt{2}z_1 z_2, z_2^2)^T$$

$$k(x, z) = \langle \phi(x), \phi(z) \rangle$$

We can see that dot product of non-linear basis function(ϕ) gives us the function of the basis vector inputs. We can compute the kernel matrix without knowing the true nature of ϕ .

Constructing Kernels:

Now let's address the elephant in the room. How do we construct the kernel. We learned above that kernel function can be any function transforming the given data into the higher dimension. Well we simply cannot use any function as a kernel function.

One of the approach to construct a valid kernel function is choose a feature space mapping $\phi(x)$ manually and then use this to find corresponding kernel as:

$$k(x_1, x_2) = \langle \phi(x_1), \phi(x_2) \rangle = \sum_{i=1}^M \phi_i(x_1)\phi_i(x_2)$$

Alternatively, we can use direct approach. We can construct kernel function K^* (suppose) directly. But we must ensure that the kernel is valid.

To do so, we have to construct a matrix using the kernels and if the matrix shows the properties of the *Gram matrix* as discussed before, then only we can use that kernel as a valid kernel.

Another popular technique to construct is to build a newer kernels based on simpler kernels. We can do this by using the following properties of the kernel:

Suppose we have two valid kernels, $k_1(x, z)$ and $k_2(x, z)$, then the following kernels will also be valid.

$$\begin{aligned} k(x, z) &= ck_1(x, z) \\ k(x, z) &= f(x)k_1(x, z)f(z) \\ k(x, z) &= q(k_1(x, z)) \\ k(x, z) &= \exp(k_1(x, z)) \\ k(x, z) &= k_1(x, z) + k_2(x, z) \\ k(x, z) &= k_1(x, z)k_2(x, z) \\ k(x, z) &= k_3(\phi(x), \phi(z)) \\ k(x, z) &= x^T A z \\ k(x, z) &= k_a(x_a, z_a) + k_b(x_b, z_b) \\ k(x, z) &= k_a(x_a, z_a)k_b(x_b, z_b) \end{aligned}$$

where $c > 0$ is a constant, $f(\cdot)$ is any function, $q(\cdot)$ is a polynomial with non-negative coefficients, $\phi(x)$ is a function from x to $\mathbb{R}^M$, $k_3(\cdot, \cdot)$ is a valid kernel in $\mathbb{R}^M$, A is a symmetric positive semidefinite matrix, x_a and x_b are variables (not necessarily disjoint) with $x = (x_a, x_b)$, and k_a and k_b are valid kernel functions over their respective spaces.

Making Predictions

We have been discussing on the kernels But once we have a valid kernel how do we make use of it to make a prediction. To do that, all we need is:

$$y = w^T \phi(x).$$

We can write this expression as:

$$w^T \phi(x) = y(\phi\phi^T)^{-1}\phi^T \phi(x) \quad (\because w = \phi(\phi\phi^T)^{-1}y)$$

$$w^T \phi(x) = y(K)^{-1}k_x$$

We replaced $\phi^T \phi(x)$ as:

$$\phi^T \phi(x) = \begin{bmatrix} \phi(x_1)^T \phi(x) \\ \phi(x_2)^T \phi(x) \\ \vdots \\ \phi(x_n)^T \phi(x) \end{bmatrix} = k_x$$

Here we see that we do not need to know the true nature of ϕ at all.

Various Kernels

There are some of the kernels that are more widely used than others. Some of the widely used off the shelf kernels are:

1. Linear Kernel

$$K(\mathbf{x}, \mathbf{z}) = \mathbf{x}^T \mathbf{z}$$

Linear kernel is also called no kernel. Kernelized algorithm using linear kernel is same as their non-kernelized counterpart.

2. Polynomial Kernel

Polynomial kernel represents the similarity of vectors over the training sample in a polynomial feature space of original vectors.

$$K(\mathbf{x}, \mathbf{z}) = (1 + \mathbf{x}^T \mathbf{z})^p$$

Here p is the degree of the polynomial. With $p = 2$, we can have a quadratic model, with $p = 3$, cubic and so on. When $p = 1$, we have a linear kernel.

It is most commonly used in natural language processing. Larger degree of p tends to overfit in NLP; hence $p = 2$ is the favoured one.

3. Radial Basis Function Kernel (RBF kernel)

RBF is a very popular kernel and is used widely in many applications. RBF kernel for some feature vector $\mathbf{x}$ and $\mathbf{z}$ is defined as

$$K(\mathbf{x}, \mathbf{z}) = e^{\left(\frac{-\|\mathbf{x} - \mathbf{z}\|^2}{2\sigma^2} \right)}$$

In the above equation, you can see the term $\|\mathbf{x} - \mathbf{z}\|^2$ which is the squared euclidian distance between $\mathbf{x}$

and $\mathbf{z}$. Hence the value of RBF kernels decrease with the distance. The range of this kernel is between 0 and 1. Before using this kernel, all the data has to be scaled.

σ is the parameter that we have to optimize. If we use larger σ then our model will underfit and if we use smaller σ , then our model will overfit. So, we have to find the sweet spot for the σ

Key Takeaways:

- We need followings to solve linearly inseparable data:
 - Multiple linear models
 - Transforming data
- Data transformation is adding dimensions to the data to make it linearly separable
- Kernel is just a data mapping function
- Using kernels is computationally cheap for data transformation.